Java MCQ Bank — Expanded Edition

Comprehensive questions with answers & explanations across Core Java, OOP, Exceptions, Collections, Lambdas, and Utilities (FileUtils, IOUtils, Random, Scanner)

Generated on Jan 18, 2026

How to use:

• **MC** = single correct • **MS** = multiple correct. Attempt first, then check the explanation.
• Use timed passes for exam practice, then slow passes for concept consolidation.

Prepared for: Java learners and interview candidates

Table of Contents

Placeholder for table of contents 0

1. What is Java?

Q1.

MC

Which statement best describes Java?

- **A.** A low-level, platform-dependent assembly language
- **B.** A high-level, object-oriented, platform-independent programming language
- **C.** A markup language for web pages
- **D.** A database query language

Answer: **B**

Explanation: Java is a high-level, object-oriented language with platform independence via the JVM and bytecode.

Q2.

MS

Select all that apply: Which features are core characteristics of Java? (Select all that apply)

- **A.** Compiled to native machine code only
- **B.** Automatic memory management (garbage collection)
- **C.** Write Once, Run Anywhere via JVM
- **D.** Multiple inheritance of implementation
- **E.** Rich standard library

Answers: **B, C, E**

Explanation: Java compiles to bytecode executed by the JVM, includes GC, and ships with a rich standard library. It does not support multiple inheritance of implementation.

Q3.

MC

What does the Java Virtual Machine (JVM) do?

- **A.** Translates Java bytecode to machine instructions at runtime
- **B.** Designs the Java API
- **C.** Stores source code
- **D.** Is the code editor for Java

Answer: **A**

Explanation: The JVM executes Java bytecode, typically using interpretation and JIT compilation.

Q4.

MC

Which of the following is true about JDK, JRE, and JVM?

- **A.** JDK includes JRE, which includes the JVM
- **B.** JRE includes JDK, which includes the JVM
- **C.** JVM includes JRE and JDK
- **D.** They are synonyms

Answer: **A**

Explanation: JDK > JRE > JVM. The JDK provides tools for development and includes the JRE, which contains the JVM.

Q5.

MS

Select all tools included in the JDK (typical). (Select all that apply)

- **A.** javac
- **B.** java
- **C.** javadoc
- **D.** mysql

Answers: **A, B, C**

Explanation: JDK includes compiler (javac), launcher (java), and javadoc; 'mysql' is unrelated.

Q6.

MC

Java source is compiled to which artifact by javac?

- **A.** .exe
- **B.** .class (bytecode)
- **C.** .dll
- **D.** .obj

Answer: **B**

Explanation: javac compiles .java to .class bytecode executed by the JVM.

Q7.

MC

Write Once, Run Anywhere (WORA) depends on which condition?

- **A.** Same CPU everywhere
- **B.** A compatible JVM on the target platform
- **C.** Using only primitive types
- **D.** Avoiding network calls

Answer: **B**

Explanation: WORA relies on a JVM available for the target OS/architecture.

Q8.

MS

Which are standard Java platforms? (Select all that apply)

- **A.** Java SE
- **B.** Java EE/Jakarta EE
- **C.** Java ME
- **D.** Java DBMS

Answers: **A, B, C**

Explanation: SE, EE (now Jakarta EE), and ME are platform editions; DBMS is not a Java platform.

—

2. Variables

Q9.

MC

Which is a valid Java variable declaration?

- **A.** int 2count = 0;
- **B.** double price = 19.99;
- **C.** String class = 'A';
- **D.** float f = 1.0;

Answer: **B**

Explanation: Identifiers cannot start with digits; 'class' is a keyword; float literal 1.0 needs 'f' suffix or cast.

Q10.

MS

Select all valid identifiers: (Select all that apply)

- **A.** _total
- **B.** $amount
- **C.** first-name
- **D.** 9lives
- **E.** value2

Answers: **A, B, E**

Explanation: Hyphen is not allowed; identifiers cannot start with a digit. '_' and '$' are allowed (though '$' is discouraged).

Q11.

MC

What is the default value of an uninitialized instance variable of type boolean?

- **A.** true
- **B.** false
- **C.** 0
- **D.** null

Answer: **B**

Explanation: For instance variables, boolean defaults to false. Local variables must be explicitly initialized.

Q12.

MC

Which statement about local variables is true?

- **A.** They are automatically initialized to default values
- **B.** They must be initialized before use
- **C.** They can be declared with access modifiers
- **D.** They persist for the life of the JVM

Answer: **B**

Explanation: Local variables have no default values and cannot use access modifiers; they live on the stack scope.

Q13.

MS

Which statements about final variables are true? (Select all that apply)

- **A.** Must be assigned exactly once
- **B.** Can be reassigned within a method
- **C.** For references, the reference is final, not the object
- **D.** Final locals can be uninitialized forever

Answers: **A, C**

Explanation: final variables are assigned once; for references, the reference is immutable but the object's state may change.

Q14.

MS

Local variable type inference (var) rules include: (Select all that apply)

- **A.** Available for local variables (Java 10+)
- **B.** Allowed for method parameters
- **C.** Allowed for fields
- **D.** Type is inferred from initializer

Answers: **A, D**

Explanation: 'var' works for local variables with initializers; not for parameters or fields.

Q15.

MC

Which assignment compiles without explicit cast?

- **A.** int i = 3.5;
- **B.** double d = 3;
- **C.** short s = 100000;
- **D.** long l = 3.0f;

Answer: **B**

Explanation: Widening from int to double is implicit; others need casts or are out of range.

Q16.

MS

Defaults apply to which? (Select all that apply)

- **A.** Instance fields
- **B.** Static fields
- **C.** Local variables
- **D.** Method parameters

Answers: **A, B**

Explanation: Only fields have default values; locals/parameters must be initialized.

—

3. Operators

Q17.

MC

What is the result of 5 + 2 * 3 in Java?

- **A.** 21
- **B.** 11
- **C.** 13
- **D.** 7

Answer: **B**

Explanation: Multiplication has higher precedence: 2*3=6; 5+6=11.

Q18.

MS

Select all true statements about == and equals() for Strings: (Select all that apply)

- **A.** '==' compares object references
- **B.** equals() compares content
- **C.** They are identical for all objects
- **D.** Using '==' on Strings may be true for interned literals

Answers: **A, B, D**

Explanation: '==' checks reference equality; equals() is content-based for String; interned literals may reference the same object.

Q19.

MC

What is the value of (10 >> 1) and (10 >>> 1)?

- **A.** 5 and 5
- **B.** 10 and 10
- **C.** 5 and 10
- **D.** Undefined

Answer: **A**

Explanation: For positive integers, arithmetic (>>) and logical (>>>) right shifts yield the same result: 5 and 5.

Q20.

MC

Which operators are short-circuit in Java?

- **A.** & and |
- **B.** && and ||
- **C.** ^ and ~
- **D.** ?: and instanceof

Answer: **B**

Explanation: Logical AND (&&) and OR (||) short-circuit; bitwise & and | do not.

Q21.

MC

Evaluate: int i=1; int x = i++ + ++i; What is x?

- **A.** 2
- **B.** 3
- **C.** 4
- **D.** 5

Answer: **C**
Explanation: i++ yields 1 then i becomes 2; ++i makes i=3 then yields 3; x=1+3=4.

Q22.
MS

Select all correct about the ternary operator a ? b : c. (Select all that apply)

- **A.** b and c must be assignable to a common type
- **B.** a must be boolean
- **C.** Ternary has lower precedence than ||
- **D.** It can be nested

Answers: **A, B, D**
Explanation: Condition must be boolean; result types must be compatible; precedence is higher than || in Java (so C is false).

Q23.
MS

Using '&' vs '&&' in conditions: (Select all that apply)

- **A.** '&' evaluates both sides
- **B.** '&&' short-circuits
- **C.** '&' is only bitwise and not logical
- **D.** Both can be used with boolean

Answers: **A, B, D**
Explanation: '&' has boolean logical form that evaluates both operands; '&&' short-circuits; both work with boolean.

Q24.
MS

For booleans, a ^ b is true when: (Select all that apply)

- **A.** Both a and b are true
- **B.** Exactly one is true
- **C.** Both are false
- **D.** a==b

Answers: **B**
Explanation: XOR yields true only when operands differ.

—

4. Strings

Q25.
MC

Which is true about Java Strings?

- **A.** They are mutable
- **B.** They are immutable
- **C.** They are primitives
- **D.** They are stored only on the stack

Answer: **B**
Explanation: String is an immutable reference type; operations produce new String objects.

Q26.
MS

Select all ways to build strings efficiently in loops: (Select all that apply)

- **A.** Using String concatenation with '+' in tight loops
- **B.** Using StringBuilder
- **C.** Using StringBuffer in single-threaded code
- **D.** Using String.format in inner loops

Answers: **B, C**
Explanation: StringBuilder is fastest in single-threaded contexts; StringBuffer is synchronized and slower. '+' in loops creates many temporary objects.

Q27.
MC

What does "Java".substring(1,3) return?

- **A.** av
- **B.** va
- **C.** Jav
- **D.** Ja

Answer: **A**
Explanation: Substring is [begin, end) so characters at indexes 1 and 2: 'a' and 'v' => 'av'.

Q28.
MC

Which statement about String pool is true?

- **A.** new String("x") uses the pool
- **B.** Literals are interned in the String pool
- **C.** String.intern() removes from pool
- **D.** The pool stores mutable strings

Answer: **B**
Explanation: String literals are interned; new String("x") creates a distinct object. intern() can add to pool, not remove.

Q29.
MS

Select all true about StringBuilder vs StringBuffer. (Select all that apply)

- **A.** StringBuffer is synchronized

- **B.** StringBuilder is typically faster in single-threaded code
- **C.** Both are immutable
- **D.** Both support append and insert

Answers: **A, B, D**
Explanation: StringBuffer is synchronized; StringBuilder is unsynchronized and faster; both are mutable and support similar APIs.

Q30.
MC

'equalsIgnoreCase' does which?

- **A.** Compares content case-insensitively
- **B.** Trims whitespace
- **C.** Normalizes accents
- **D.** Ignores punctuation

Answer: **A**
Explanation: equalsIgnoreCase ignores ASCII case; it doesn't trim or normalize.

Q31.
MC

What does 'Java'.compareTo('Jar') return sign-wise?

- **A.** Negative
- **B.** Zero
- **C.** Positive
- **D.** Throws

Answer: **C**
Explanation: Lexicographically, 'v' > 'r', so positive.

Q32.
MS

Select all that apply to String APIs (Java 8+). (Select all that apply)

- **A.** String.join can join with delimiter
- **B.** String.repeat is available in Java 11+
- **C.** String.strip is available in Java 11+
- **D.** String is a value type

Answers: **A, B, C**
Explanation: join, repeat(11+), and strip(11+) exist; String is a reference type, not a value type.

—

5. Control Flow

Q33.
MC

What does a switch statement in Java require for case labels?

- **A.** Non-constant expressions

- **B.** Constant expressions compatible with the switch expression type
- **C.** Only Strings are allowed
- **D.** Only integers are allowed

Answer: **B**

Explanation: Case labels must be compile-time constants of a type compatible with the switch expression (int, enum, String, etc.).

Q34.

MS

Select all that compile for switch expressions (Java 14+): (Select all that apply)

- **A.** yield to return a value
- **B.** Multiple labels per case, e.g., case 1,2 ->
- **C.** Switch on long prior to Java 7
- **D.** Arrow labels without break

Answers: **A, B, D**

Explanation: Switch expressions support yield, multiple labels, and arrow syntax; long was not supported before Java 7, and classic switch used 'break'.

Q35.

MC

What is the effect of 'break' in a loop?

- **A.** Skips current iteration
- **B.** Exits the loop
- **C.** Exits the method
- **D.** Pauses the loop

Answer: **B**

Explanation: 'break' exits the nearest enclosing loop or switch.

Q36.

MC

What does 'continue' do in a for loop?

- **A.** Terminates the loop
- **B.** Skips to next iteration
- **C.** Jumps to labeled block outside loop
- **D.** Throws an exception

Answer: **B**

Explanation: 'continue' skips the rest of the current iteration and proceeds with the next one.

Q37.

MS

Classic switch fall-through occurs when: (Select all that apply)

- **A.** Missing break in block-style cases
- **B.** Using arrow labels (->)
- **C.** Using yield

- **D.** Having consecutive cases without code

Answers: **A, D**
Explanation: Block-style cases without break fall through; empty case labels can fall through to next with code.

Q38.
MC

What does a labeled continue do?

- **A.** Exits all loops
- **B.** Continues the labeled outer loop
- **C.** Continues current loop only
- **D.** Throws an exception

Answer: **B**
Explanation: continue label jumps to next iteration of the labeled loop.

Q39.
MC

Does 'if((x = 5) > 0)' compile?

- **A.** Yes
- **B.** No

Answer: **A**
Explanation: Assignment is allowed in expressions; the result (5) is compared.

Q40.
MS

Which types are valid for switch (classic or expressions) in modern Java? (Select all that apply)

- **A.** int/byte/short/char
- **B.** enum
- **C.** String
- **D.** long in classic switch pre-Java 7

Answers: **A, B, C**
Explanation: int/byte/short/char, enums, and String are valid; long wasn't supported in classic switch before Java 7.

—

6. Arrays

Q41.
MC

Which is a correct array declaration and initialization?

- **A.** int arr[5] = new int[5];
- **B.** int[] arr = {1,2,3};
- **C.** int arr = new int[]{1,2,3};
- **D.** int[] arr = new int[3]{1,2,3};

Answer: **B**

Explanation: Java uses int[] arr; array initializer syntax {..} is valid at declaration; sizes are not specified with initializer.

Q42.

MS

Select all true about arrays: (Select all that apply)

- **A.** Arrays know their length at runtime via length field
- **B.** Arrays can have negative length
- **C.** Arrays are covariant
- **D.** ArrayIndexOutOfBoundsException is unchecked

Answers: **A, C, D**

Explanation: Arrays expose 'length'; negative sizes are illegal; arrays are covariant (e.g., String[] is a subtype of Object[]); AIOOBE is RuntimeException.

Q43.

MC

What is the default value for elements of a new int[3]?

- **A.** null
- **B.** 0
- **C.** 1
- **D.** undefined

Answer: **B**

Explanation: Numeric arrays are zero-initialized.

Q44.

MC

Which creates a 2D array with 2 rows and 3 columns?

- **A.** int[][] a = new int[2,3];
- **B.** int[][] a = new int[2][3];
- **C.** int[2][3] a = new int;
- **D.** int[][] a = {2,3};

Answer: **B**

Explanation: Java uses separate brackets per dimension: new int[rows][cols].

Q45.

MS

Which statements are true? (Select all that apply)

- **A.** Arrays can have length 0
- **B.** 'length' is a field, not a method
- **C.** Arrays implement List
- **D.** Arrays are objects

Answers: **A, B, D**

Explanation: Zero-length arrays are allowed; length is a field; arrays are objects but they don't implement List.

Q46.

MC

What does Arrays.asList(int[]) return?

- **A.** List with elements
- **B.** List with one element
- **C.** Compilation error
- **D.** Empty list

Answer: **B**

Explanation: Primitives are not auto-boxed into a var-args list; the whole int[] becomes a single element.

Q47.

MS

System.arraycopy vs Arrays.copyOf: (Select all that apply)

- **A.** arraycopy copies between existing arrays
- **B.** copyOf allocates a new array
- **C.** copyOf can change length
- **D.** arraycopy resizes destination

Answers: **A, B, C**

Explanation: arraycopy moves data between arrays; copyOf allocates new array with specified length.

Q48.

MC

What is printed? int[] a={1,2}; for(int x : a){ x++; } System.out.println(a[0]);

- **A.** 1
- **B.** 2
- **C.** 3
- **D.** Compilation error

Answer: **A**

Explanation: Enhanced for assigns element value to a copy; incrementing 'x' doesn't mutate the array.

—

7. Loops

Q49.

MC

What is printed by: for(int i=0;i<3;i++){ if(i==1) continue; System.out.print(i); }

- **A.** 012
- **B.** 02
- **C.** 12
- **D.** 01

Answer: **B**

Explanation: When i==1, continue skips printing; outputs 0 then 2 => '02'.

Q50.

MS

Select all valid enhanced for uses: (Select all that apply)

- **A.** for(int x : new int[]{1,2,3})
- **B.** for(var s : list)
- **C.** for(String k, v : map)
- **D.** for(Map.Entry e : map.entrySet())

Answers: **A, B, D**

Explanation: Enhanced for iterates over arrays or Iterable; maps require iterating over entrySet or keySet; no tuple syntax.

Q51.

MC

Which loop guarantees the body executes at least once?

- **A.** for
- **B.** while
- **C.** do-while
- **D.** enhanced for

Answer: **C**

Explanation: do-while evaluates the condition after executing the body.

Q52.

MC

What does a labeled break do?

- **A.** Exits only inner loop
- **B.** Exits to the labeled statement outside nested loops
- **C.** Skips to next iteration of inner loop
- **D.** Throws an exception

Answer: **B**

Explanation: labeled break breaks out to the label target, useful in nested loops.

Q53.

MS

Which is a valid infinite loop? (Select all that apply)

- **A.** for(;;)
- **B.** while(true)
- **C.** do{}while(false);
- **D.** for(int i=0;i<1;i--)

Answers: **A, B, D**

Explanation: for(;;) and while(true) are classic infinite loops; a decrementing loop with i-- and condition i<1 also doesn't terminate.

Q54.

MC

Modifying a List during enhanced for without iterator.remove():

- **A.** Is safe

- **B.** May throw ConcurrentModificationException
- **C.** Always compiles but may fail at runtime
- **D.** Never compiles

Answer: **B, C**
Explanation: Structural modification during iteration typically triggers ConcurrentModificationException at runtime.

Q55.
MC

What does this print? int i=0; while(i++<2){ } System.out.print(i);

- **A.** 1
- **B.** 2
- **C.** 3
- **D.** 0

Answer: **C**
Explanation: Post-increment compares old value, loop runs while 0<2 and 1<2; after loop i==3.

Q56.
MS

ListIterator can: (Select all that apply)

- **A.** iterate backwards
- **B.** insert during iteration
- **C.** set last returned element
- **D.** iterate arrays

Answers: **A, B, C**
Explanation: ListIterator supports previous(), add(), and set(); it doesn't iterate arrays directly.

—

8. Classes and Objects

Q57.
MC

Which is true about constructors?

- **A.** Constructors have a return type
- **B.** Default constructor is provided if no constructor exists
- **C.** Constructors can be inherited
- **D.** Constructors can be abstract

Answer: **B**
Explanation: Java provides a default no-arg constructor only if no constructors are declared; constructors have no return type and are not inherited.

Q58.
MS

Select all correct statements about 'this': (Select all that apply)

- **A.** Refers to current object
- **B.** Can call another constructor via this(...)
- **C.** Can be used in static context
- **D.** Used to disambiguate fields from params

Answers: **A, B, D**
Explanation: 'this' refers to the current instance; this(...) chains constructors; it cannot be used in static context.

Q59.
MC

What is object composition?

- **A.** A class inheriting fields from a parent
- **B.** Building complex types by including references to other objects
- **C.** Overriding methods at runtime
- **D.** Using static fields

Answer: **B**
Explanation: Composition models 'has-a' relationships by composing objects.

Q60.
MC

Which modifiers are valid for a top-level class?

- **A.** public
- **B.** protected
- **C.** private
- **D.** package-private (no modifier)

Answer: **A, D**
Explanation: Top-level classes can be public or package-private only.

Q61.
MS

Initialization order (Select all correct)

- **A.** Static fields/blocks in declaration order (once)
- **B.** Instance fields/blocks before constructor
- **C.** Constructor runs after instance init
- **D.** Constructor runs before instance init

Answers: **A, B, C**
Explanation: Static init first (class load), then for each new object: instance init, then constructor.

Q62.
MS

Overload resolution prefers: (Select all that apply)

- **A.** Exact match over boxing
- **B.** Boxing over varargs
- **C.** Varargs over exact match

- **D.** Widening over boxing

Answers: **A, B, D**
Explanation: Exact > widening > boxing > varargs (simplified).

Q63.
MS

Static nested vs inner class: (Select all that apply)

- **A.** Static nested has no implicit outer reference
- **B.** Inner class requires an instance of outer
- **C.** Static nested can have static members
- **D.** Inner class can declare static members freely

Answers: **A, B, C**
Explanation: Inner classes cannot declare static members except constants; static nested has no implicit outer ref and may have statics.

Q64.
MS

Anonymous classes can: (Select all that apply)

- **A.** Extend one class
- **B.** Implement one interface
- **C.** Have explicit constructors
- **D.** Have instance initializers

Answers: **A, B, D**
Explanation: Anonymous classes have no named constructors but can use instance initializer blocks.

—

9. Inheritance

Q65.
MC

Java supports which kind of inheritance for classes?

- **A.** Multiple inheritance of classes
- **B.** Single inheritance of classes
- **C.** No inheritance
- **D.** Multiple inheritance through classes and interfaces

Answer: **B**
Explanation: Java supports single inheritance for classes and multiple inheritance of type through interfaces.

Q66.
MS

Select all correct about method overriding: (Select all that apply)

- **A.** Return type must be identical or covariant
- **B.** Access can be more restrictive
- **C.** @Override helps catch mistakes

- **D.** Static methods can be overridden

Answers: **A, C**
Explanation: Overriding can widen access, not restrict; static methods are hidden, not overridden; covariant returns allowed.

Q67.
MC
Which keyword calls the parent constructor?

- **A.** parent()
- **B.** base()
- **C.** super()
- **D.** this()

Answer: **C**
Explanation: super() invokes the immediate superclass constructor.

Q68.
MC
What happens if a subclass does not call super() explicitly?

- **A.** Compilation error always
- **B.** The no-arg super() is inserted if available
- **C.** this() is inserted instead
- **D.** Runtime exception

Answer: **B**
Explanation: The compiler inserts an implicit no-arg super() as the first statement if the superclass has it.

Q69.
MS
Select all that compile regarding overriding: (Select all that apply)

- **A.** Overriding method can widen access
- **B.** Overriding method can throw broader checked exceptions
- **C.** Return type can be covariant
- **D.** Static methods are overridden

Answers: **A, C**
Explanation: Access can be widened, return can be covariant; checked exceptions cannot be broader; static methods are hidden, not overridden.

Q70.
MC
Order of constructor calls when new Sub():

- **A.** Subclass first then superclass
- **B.** Superclass first then subclass
- **C.** Random
- **D.** Based on field order only

Answer: **B**

Explanation: Superclass constructor executes before subclass constructor.

Q71.

MS

instanceof rules: which are true? (Select all that apply)

- **A.** null instanceof Type is false
- **B.** An object is instanceof its class and superclasses
- **C.** Interfaces can be used with instanceof
- **D.** It checks compile-time only

Answers: **A, B, C**

Explanation: instanceof is a runtime check; null is never instanceof anything.

Q72.

MC

Calling super.method() from subclass does what?

- **A.** Calls overridden parent method
- **B.** Calls child method again
- **C.** Is illegal
- **D.** Calls interface default method

Answer: **A**

Explanation: super.method() invokes the parent's implementation.

—

10. Abstraction

Q73.

MS

Which statements about abstract classes are true? (Select all that apply)

- **A.** They can have implemented methods
- **B.** They can be instantiated directly
- **C.** They can declare constructors
- **D.** All methods must be abstract

Answers: **A, C**

Explanation: Abstract classes can have concrete methods and constructors; they cannot be instantiated; not all methods need be abstract.

Q74.

MC

An interface in Java (pre-Java 8) can contain which of the following?

- **A.** Abstract methods
- **B.** Instance fields
- **C.** Constructors
- **D.** Static initializers

Answer: **A**

Explanation: Pre-Java 8 interfaces contained abstract methods and constants (public static final). No constructors or instance fields.

Q75.

MS

Since Java 8+, interfaces can have: (Select all that apply)

- **A.** default methods
- **B.** static methods
- **C.** private methods
- **D.** instance fields

Answers: **A, B, C**

Explanation: Java 8 added default and static methods; Java 9 added private methods. Interfaces still cannot have instance fields (only constants).

Q76.

MC

Which keyword prevents a class from being subclassed?

- **A.** final
- **B.** sealed
- **C.** abstract
- **D.** static

Answer: **A**

Explanation: 'final' classes cannot be extended. (Sealed classes restrict, not prevent, extension.)

Q77.

MS

Which are true of interfaces? (Select all that apply)

- **A.** Fields are implicitly public static final
- **B.** Methods are implicitly public abstract (unless default/static/private)
- **C.** Can have constructors
- **D.** Can extend multiple interfaces

Answers: **A, B, D**

Explanation: Interfaces have constants, can extend multiple interfaces, and methods default to public abstract unless otherwise declared.

Q78.

MS

Abstract class rules: (Select all that apply)

- **A.** May contain no abstract methods
- **B.** Cannot be instantiated
- **C.** Must define all constructors of subclasses
- **D.** Can have state

Answers: **A, B, D**

Explanation: Abstract classes can be purely concrete in methods, cannot be instantiated, and can hold state.

Q79.

MS

Sealed classes (Java 17+) require: (Select all that apply)

- **A.** A permits clause listing subclasses
- **B.** Subclasses must be final, sealed, or non-sealed
- **C.** All subclasses must be in same package
- **D.** Prevent all extensions globally

Answers: **A, B**

Explanation: Sealed uses permits and requires specific subclass modifiers; they restrict, not fully prevent, extension.

Q80.

MC

Default method conflict resolution: if two interfaces provide same default

- **A.** Compilation error unless overridden
- **B.** Automatically picks first
- **C.** Picks lexicographically smaller
- **D.** Runtime error

Answer: **A**

Explanation: Implementing class must override to resolve ambiguity.

—

11. Encapsulation

Q81.

MC

Encapsulation is primarily achieved by:

- **A.** public fields
- **B.** private fields with getters/setters
- **C.** global variables
- **D.** multiple inheritance

Answer: **B**

Explanation: Encapsulation hides internal state behind accessors/mutators, controlling access.

Q82.

MS

Select all good practices for encapsulation: (Select all that apply)

- **A.** Expose mutable internals directly
- **B.** Return defensive copies of mutable fields
- **C.** Use immutable objects when possible
- **D.** Use public fields for everything

Answers: **B, C**

Explanation: Defensive copying and immutability preserve encapsulation; exposing mutables or public fields breaks it.

Q83.

MC

What is the effect of making a class immutable?

- **A.** Thread-safety benefits
- **B.** No need for synchronization for reads
- **C.** State can change post-construction
- **D.** Objects can be used as map keys safely

Answer: **A, B, D**

Explanation: Immutable objects are inherently thread-safe, don't require synchronization for reads, and are safe as keys if equals/hashCode are consistent.

Q84.

MC

Which is NOT a step toward immutability?

- **A.** Make fields final
- **B.** Provide setters for each field
- **C.** Do not expose internals
- **D.** Initialize all fields in constructor

Answer: **B**

Explanation: Setters permit mutation, contradicting immutability.

Q85.

MS

Best way to expose internal List safely? (Select all that apply)

- **A.** Return Collections.unmodifiableList(view)
- **B.** Return defensive copy
- **C.** Return direct modifiable list
- **D.** Expose array field

Answers: **A, B**

Explanation: Unmodifiable views or defensive copies protect invariants; exposing mutables breaks encapsulation.

Q86.

MS

Java records (Java 16+) are: (Select all that apply)

- **A.** Implicitly final
- **B.** Provide canonical constructor
- **C.** Guarantee deep immutability
- **D.** Provide equals/hashCode/toString

Answers: **A, B, D**

Explanation: Records are final, synthesize canonical constructor and equals/hashCode/toString; deep immutability depends on component types.

Q87.

MS

Getter/setter pitfalls include: (Select all that apply)

- **A.** Returning internal mutable state
- **B.** Setters that break invariants
- **C.** Using Optional in setters always
- **D.** Leaking references via collections

Answers: **A, B, D**

Explanation: Avoid leaking mutable internals and avoid setters that violate invariants.

Q88.

MC

Which promotes immutability?

- **A.** Make fields final
- **B.** Provide with-methods returning new instance
- **C.** Expose public fields
- **D.** Allow subclassing freely

Answer: **A, B**

Explanation: Final fields and with-methods help sustain immutability; public fields and easy subclassing work against it.

—

12. Access Modifiers

Q89.

MC

Which access level allows visibility within the same package and subclasses in other packages?

- **A.** private
- **B.** default (package-private)
- **C.** protected
- **D.** public

Answer: **C**

Explanation: protected members are visible in the same package and to subclasses even across packages.

Q90.

MS

Select all correct about top-level access: (Select all that apply)

- **A.** Top-level classes can be private
- **B.** Only one public class per file
- **C.** A file can contain multiple package-private classes
- **D.** Top-level class name must match file name if public

Answers: **B, C, D**
Explanation: Only one public class per file; multiple package-private classes allowed; public class name must match file name; no private top-level classes.

Q91.
MC

What does private mean for a field?

- **A.** Visible only within its class
- **B.** Visible within package
- **C.** Visible to subclasses
- **D.** Visible everywhere

Answer: **A**
Explanation: private restricts access to the declaring class only.

Q92.
MC

Which access modifier provides the least restriction?

- **A.** public
- **B.** protected
- **C.** default (package-private)
- **D.** private

Answer: **A**
Explanation: public is the least restrictive.

Q93.
MS

In a subclass in a different package, protected member access is allowed via: (Select all that apply)

- **A.** Subclass instance reference
- **B.** Superclass reference
- **C.** Any instance of the subclass
- **D.** Unrelated class instance

Answers: **A, C**
Explanation: Protected across packages is allowed through the subclass type, not via arbitrary superclass references.

Q94.
MS

Interface members without modifiers are: (Select all that apply)

- **A.** public methods
- **B.** private fields
- **C.** public static final fields
- **D.** package-private methods

Answers: **A, C**
Explanation: Interface methods default to public abstract; fields are public static final.

Q95.

MS

Private constructors are used for: (Select all that apply)

- **A.** Singleton pattern
- **B.** Static factory classes
- **C.** Preventing instantiation
- **D.** Enabling inheritance

Answers: **A, B, C**

Explanation: Private constructors restrict instantiation; often used in utility classes or singletons.

Q96.

MC

Default (package-private) permits access in:

- **A.** Same package
- **B.** Subclasses in other packages
- **C.** Everywhere
- **D.** Nowhere

Answer: **A**

Explanation: Package-private visibility is limited to the same package.

—

13. Polymorphism

Q97.

MC

Polymorphism in Java primarily refers to:

- **A.** Method overloading only
- **B.** Method overriding and dynamic dispatch
- **C.** Multiple inheritance
- **D.** Operator overloading

Answer: **B**

Explanation: Runtime polymorphism via overriding and dynamic dispatch is central to Java's polymorphism.

Q98.

MS

Select all true about overloading vs overriding: (Select all that apply)

- **A.** Overloading is compile-time
- **B.** Overriding is runtime
- **C.** Overloading requires different parameter lists
- **D.** Overriding can change parameter types arbitrarily

Answers: **A, B, C**

Explanation: Overloading resolves at compile-time and needs a different signature; overriding must keep the same signature (with covariant return).

Q99.

MC

What is the output? Object o = "hi"; System.out.println(o.toString());

- **A.** Compilation error
- **B.** hi
- **C.** Reference address
- **D.** null

Answer: **B**

Explanation: toString() is dynamically dispatched and String.toString() returns the string itself.

Q100.

MC

Which allows treating objects of different classes uniformly?

- **A.** Encapsulation
- **B.** Polymorphism via common interfaces
- **C.** Static methods
- **D.** Private constructors

Answer: **B**

Explanation: Polymorphism allows multiple implementations behind a common interface or superclass.

Q101.

MS

Fields vs methods: (Select all that apply)

- **A.** Fields are not polymorphic
- **B.** Methods are polymorphic via overriding
- **C.** Static methods are chosen at runtime
- **D.** Instance methods are chosen at runtime

Answers: **A, B, D**

Explanation: Fields are hidden (not polymorphic); instance methods dispatch dynamically; static methods are resolved at compile time.

Q102.

MS

Downcasting risks: (Select all that apply)

- **A.** ClassCastException at runtime
- **B.** Compilation error always
- **C.** Requires instanceof or knowledge
- **D.** Prevents polymorphism

Answers: **A, C**

Explanation: Downcasts may throw at runtime; use instanceof or designs avoiding downcasts.

Q103.

MC

Covariant returns allow:

- **A.** Changing return type to a subtype
- **B.** Changing parameter types
- **C.** Changing thrown exceptions arbitrarily
- **D.** Changing method name

Answer: **A**
Explanation: Covariant return types allow subtype return; parameters/signature must match.

Q104.
MC

Static method hiding occurs when:

- **A.** Subclass defines static method with same signature
- **B.** Runtime picks child method
- **C.** Compile-time binds to reference type
- **D.** It's illegal

Answer: **A, C**
Explanation: Static methods are hidden, not overridden; resolution is compile-time based on reference type.

—

14. Exception Handling

Q105.
MS

Which are checked exceptions? (Select all that apply)

- **A.** IOException
- **B.** SQLException
- **C.** NullPointerException
- **D.** FileNotFoundException

Answers: **A, B, D**
Explanation: IOException, SQLException, FileNotFoundException are checked; NPE is unchecked (RuntimeException).

Q106.
MC

What happens if a checked exception is neither caught nor declared?

- **A.** Runtime error
- **B.** Compilation error
- **C.** Silently ignored
- **D.** Converted to unchecked

Answer: **B**
Explanation: Java requires checked exceptions to be caught or declared in the method signature.

Q107.
MS

Select all true about try-with-resources: (Select all that apply)

- **A.** Resources must implement AutoCloseable
- **B.** Suppressed exceptions can be retrieved via getSuppressed()
- **C.** Resources close in LIFO order
- **D.** Only works with Reader/Writer

Answers: **A, B, C**
Explanation: Any AutoCloseable works; suppressed exceptions are recorded; closing order is reverse of acquisition.

Q108.
MC

What does finally guarantee?

- **A.** Always executes regardless of return/throw (barring System.exit or fatal error)
- **B.** Executes only when no exception
- **C.** Executes only on exception
- **D.** Executes before try

Answer: **A**
Explanation: finally runs after try/catch whether or not an exception occurs, except in extreme cases like System.exit.

Q109.
MC

Order of catch blocks should be:

- **A.** Subclass first, then superclass
- **B.** Superclass first, then subclass
- **C.** Any order
- **D.** Checked after unchecked

Answer: **A**
Explanation: More specific first to avoid unreachable code.

Q110.
MC

Multi-catch (catch(A|B e)) variable:

- **A.** Is effectively final
- **B.** Can be reassigned
- **C.** Must be Throwable
- **D.** Cannot be used

Answer: **A**
Explanation: In multi-catch, the caught exception variable is effectively final.

Q111.
MS

Throw vs throws: (Select all that apply)

- **A.** throw uses an exception instance
- **B.** throws declares potential exceptions

- **C.** throws at call site
- **D.** throw inside method body

Answers: **A, B, D**
Explanation: 'throw' actually throws; 'throws' declares in the method signature.

Q112.
MC

Suppressed exceptions appear when:

- **A.** try-with-resources closes resources
- **B.** Exceptions during close are added via getSuppressed()
- **C.** finally throws
- **D.** No exception ever

Answer: **A, B**
Explanation: Suppressed exceptions are captured from close operations in try-with-resources.

—

15. Collections

Q113.
MS

Which interfaces are part of the Java Collections Framework root hierarchy? (Select all that apply)

- **A.** Collection
- **B.** Map
- **C.** List
- **D.** Set

Answers: **A, B, C, D**
Explanation: Collection is a root for List/Set/Queue; Map is a separate root.

Q114.
MC

Which collection allows duplicate elements?

- **A.** Set
- **B.** List
- **C.** Map
- **D.** EnumSet

Answer: **B**
Explanation: Lists allow duplicates and preserve insertion order; Sets disallow duplicates.

Q115.
MS

Select all correct about fail-fast iterators: (Select all that apply)

- **A.** They throw ConcurrentModificationException on structural modification
- **B.** They are guaranteed in all cases
- **C.** They are best-effort and not guaranteed

- **D.** Iterating and modifying via iterator.remove() is safe

Answers: **A, C, D**
Explanation: Fail-fast is best-effort; iterator.remove() is the supported modification during iteration.

Q116.
MC

Which is NOT in java.util?

- **A.** ArrayList
- **B.** HashMap
- **C.** Hashtable
- **D.** CopyOnWriteArrayList

Answer: **D**
Explanation: CopyOnWriteArrayList is in java.util.concurrent; the others are in java.util (Hashtable is legacy but in java.util).

Q117.
MS

Queue/Deque operations: (Select all that apply)

- **A.** offer/poll/peek do not throw on failure
- **B.** add/remove/element may throw
- **C.** Deque supports addFirst/addLast
- **D.** Queue supports push/pop

Answers: **A, B, C**
Explanation: offer/poll/peek return special values; add/remove/element may throw; Deque has head/tail ops; push/pop are Deque methods, not Queue.

Q118.
MC

Collections.unmodifiableList(list):

- **A.** Is a view over 'list'
- **B.** Reflects changes in underlying list
- **C.** Prevents structural mutation via the view
- **D.** Copies data

Answer: **A, B, C**
Explanation: Unmodifiable views reflect underlying changes but prevent mutation through the view.

Q119.
MC

Arrays.asList(...) returns:

- **A.** A fixed-size list backed by the array
- **B.** A resizable ArrayList
- **C.** An immutable list
- **D.** A deep copy

Answer: **A**

Explanation: The returned list has fixed size and is backed by the array.

Q120.

MS

Comparator contract best practices: (Select all that apply)

- **A.** Be consistent with equals when possible
- **B.** Ensure transitivity
- **C.** Allow compare(a,b)==0 yet !a.equals(b)
- **D.** Use Comparator.thenComparing for tie-breakers

Answers: **A, B, D**

Explanation: Comparators should be transitive; consistency with equals avoids surprises; chaining helps tie-breakers.

—

16. Collections - Lists

Q121.

MS

ArrayList vs LinkedList: which statements are true? (Select all that apply)

- **A.** ArrayList has O(1) random access
- **B.** LinkedList has O(1) random access
- **C.** LinkedList is better for frequent insertions/removals in the middle
- **D.** ArrayList resizing may cause reallocation

Answers: **A, C, D**

Explanation: ArrayList provides fast random access; LinkedList does not; LinkedList is efficient for middle insertions/removals; ArrayList may resize by growing array.

Q122.

MC

What is the default initial capacity of an ArrayList (Java 8+)?

- **A.** 0 until first add triggers 10
- **B.** 10 immediately
- **C.** 16
- **D.** 1

Answer: **A**

Explanation: In Java 8+, empty ArrayList uses a shared empty array and grows to 10 on first add.

Q123.

MC

Which method removes and returns the element at an index?

- **A.** get(int index)
- **B.** remove(int index)
- **C.** poll()
- **D.** delete(int index)

Answer: **B**
Explanation: remove(int) removes and returns the element at the specified index.

Q124.
MS
Which operations are supported by List.of(...) immutable lists? (Select all that apply)

- **A.** add
- **B.** set
- **C.** get
- **D.** contains

Answers: **C, D**
Explanation: List.of returns an unmodifiable list; read-only operations like get/contains work; mutators throw UnsupportedOperationException.

Q125.
MS
subList behavior: (Select all that apply)

- **A.** Is a view backed by original list
- **B.** Structural changes in parent may invalidate it
- **C.** Modifying subList affects parent
- **D.** Independent copy

Answers: **A, B, C**
Explanation: subList is a view; modifications reflect both ways; concurrent structural changes can cause issues.

Q126.
MC
Vector characteristics:

- **A.** Legacy synchronized list
- **B.** Part of java.util
- **C.** Always faster than ArrayList
- **D.** Supports capacityIncrement

Answer: **A, B, D**
Explanation: Vector is legacy, synchronized, has capacity increment; not necessarily faster.

Q127.
MS
CopyOnWriteArrayList (Select all that apply)

- **A.** Iterators are snapshot-based
- **B.** Writes copy the whole array
- **C.** Great for many writes
- **D.** Good for many reads, few writes

Answers: **A, B, D**
Explanation: Copy-on-write suits read-heavy scenarios; writes are expensive.

Q128.

MS

ListIterator differences from Iterator: (Select all that apply)

- **A.** Supports bidirectional traversal
- **B.** Has add and set methods
- **C.** Works for Set
- **D.** Index aware

Answers: **A, B, D**

Explanation: ListIterator is bidirectional, can add/set, and tracks indices; it's for lists.

—

17. Collections - Sets

Q129.

MC

Which set implementation maintains insertion order?

- **A.** HashSet
- **B.** TreeSet
- **C.** LinkedHashSet
- **D.** EnumSet

Answer: **C**

Explanation: LinkedHashSet maintains insertion order; HashSet is unordered; TreeSet sorts; EnumSet is specialized.

Q130.

MS

Select all true about TreeSet: (Select all that apply)

- **A.** Requires elements to be Comparable or a Comparator
- **B.** Allows null elements reliably
- **C.** Is sorted
- **D.** Backed by a hash table

Answers: **A, C**

Explanation: TreeSet is a NavigableSet sorted by natural/comparator order; it is typically backed by a balanced tree and does not reliably allow nulls.

Q131.

MS

Which operations are O(1) average-case for HashSet? (Select all that apply)

- **A.** add
- **B.** contains
- **C.** remove
- **D.** iteration

Answers: **A, B, C**

Explanation: HashSet add/contains/remove are O(1) average; iteration is O(n).

Q132.

MC

Which set type is optimized for enum keys?

- **A.** EnumSet
- **B.** HashSet
- **C.** TreeSet
- **D.** CopyOnWriteArraySet

Answer: **A**

Explanation: EnumSet uses bit vectors internally and is very efficient for enum types.

Q133.

MS

equals/hashCode in sets: (Select all that apply)

- **A.** Must be consistent for correct behavior
- **B.** HashSet relies on hashCode
- **C.** Violations may allow duplicates logically
- **D.** TreeSet ignores equals/hashCode

Answers: **A, B, C, D**

Explanation: HashSet uses equals/hashCode; TreeSet relies on compareTo/Comparator, potentially ignoring equals/hashCode consistency.

Q134.

MS

Null handling: (Select all that apply)

- **A.** HashSet allows one null element
- **B.** TreeSet may throw on null without Comparator allowing it
- **C.** EnumSet allows null
- **D.** LinkedHashSet allows null

Answers: **A, B, D**

Explanation: EnumSet doesn't allow null; HashSet/LinkedHashSet allow single null; TreeSet typically disallows.

Q135.

MC

Set.of(...) (Java 9+):

- **A.** Returns unmodifiable set
- **B.** Rejects duplicates at creation
- **C.** Allows null
- **D.** Maintains insertion order

Answer: **A, B**

Explanation: Set.of is unmodifiable and rejects duplicates; doesn't allow null; order is unspecified.

Q136.

MS

NavigableSet (Select all that apply)

- **A.** Provides floor/ceiling
- **B.** Provides lower/higher
- **C.** Is implemented by TreeSet
- **D.** Is always hash-based

Answers: **A, B, C**
Explanation: TreeSet implements NavigableSet and supports navigation methods.

—

18. Collections - Maps

Q137.
MC

Which Map maintains insertion order?

- **A.** HashMap
- **B.** LinkedHashMap
- **C.** TreeMap
- **D.** Hashtable

Answer: **B**
Explanation: LinkedHashMap preserves insertion order; TreeMap sorts; HashMap is unordered.

Q138.
MS

Select all true about HashMap (Java 8+): (Select all that apply)

- **A.** Buckets may treeify into red-black trees
- **B.** Allows one null key
- **C.** Is synchronized by default
- **D.** Iteration order is undefined

Answers: **A, B, D**
Explanation: HashMap can treeify bins for performance; allows one null key; not synchronized; order is unspecified.

Q139.
MC

Which Map is sorted by keys?

- **A.** HashMap
- **B.** LinkedHashMap
- **C.** TreeMap
- **D.** WeakHashMap

Answer: **C**
Explanation: TreeMap sorts by natural or comparator order of keys.

Q140.
MS

Which statements about ConcurrentHashMap are true? (Select all that apply)

- **A.** Allows null keys/values
- **B.** Thread-safe concurrent access
- **C.** Iterators are weakly consistent
- **D.** All operations are blocking

Answers: **B, C**
Explanation: ConcurrentHashMap is thread-safe, disallows nulls, and its iterators are weakly consistent; many operations are non-blocking.

Q141.
MC

IdentityHashMap uses:

- **A.** Reference equality (==)
- **B.** equals()
- **C.** hashCode() of content
- **D.** Weak references

Answer: **A**
Explanation: IdentityHashMap compares keys by reference equality.

Q142.
MS

WeakHashMap keys: (Select all that apply)

- **A.** Are wrapped in weak references
- **B.** Eligible for GC when no strong refs
- **C.** Values are weak too
- **D.** Ideal for caches

Answers: **A, B, D**
Explanation: Weak keys allow GC-based cleanup; values remain strong unless you manage otherwise; often used for caches.

Q143.
MC

Map.getOrDefault(k,def):

- **A.** Returns def if key absent
- **B.** Inserts def
- **C.** Returns null only if mapped to null
- **D.** Never returns null

Answer: **A, C**
Explanation: It doesn't insert def; returns default only when absent; null may be returned if key maps to null.

Q144.
MC

computeIfAbsent does what?

- **A.** Computes a value if key missing
- **B.** May be called multiple times per key in concurrency

- **C.** Inserts the computed value
- **D.** Throws if key present

Answer: **A, C**
Explanation: computeIfAbsent computes and inserts when missing; in concurrent maps computation may be invoked once per insertion.

—

19. Lambda Functions

Q145.
MC
What is a functional interface?

- **A.** An interface with exactly one abstract method
- **B.** Any interface
- **C.** A class with one method
- **D.** An interface with only default methods

Answer: **A**
Explanation: Functional interfaces have a single abstract method (SAM); they may also have default/static methods.

Q146.
MS
Select all valid lambda expressions for (int x, int y) -> int: (Select all that apply)

- **A.** (x,y) -> x + y
- **B.** x,y -> { return x + y; }
- **C.** (int x, int y) -> { return x + y; }
- **D.** (x, y) -> { x + y; }

Answers: **A, C**
Explanation: Parameters need parentheses unless a single inferred type parameter; 'return' required in block with explicit return; expression body works. Option B missing parentheses; D lacks return.

Q147.
MS
Which functional interfaces are in java.util.function? (Select all that apply)

- **A.** Predicate
- **B.** Function
- **C.** Comparator
- **D.** Supplier

Answers: **A, B, D**
Explanation: Predicate, Function, Supplier are in java.util.function; Comparator is in java.util but also a functional interface.

Q148.
MC
What does method reference String::length represent?

- **A.** A reference to static method length
- **B.** A reference to an instance method of a particular object
- **C.** A reference to an instance method of an arbitrary object of type String
- **D.** A constructor reference

Answer: **C**
Explanation: String::length references the instance method length() on an arbitrary String instance.

Q149.
MC

Effectively final means:

- **A.** Local variable not modified after assignment
- **B.** final keyword required
- **C.** Lambdas can capture it
- **D.** It lives on heap

Answer: **A, C**
Explanation: Lambdas capture variables that are final or effectively final.

Q150.
MC

Inside a lambda, 'this' refers to:

- **A.** The lambda itself
- **B.** The enclosing instance
- **C.** A new anonymous object
- **D.** null

Answer: **B**
Explanation: 'this' inside a lambda refers to the enclosing class instance, unlike anonymous classes.

Q151.
MS

Method references: (Select all that apply)

- **A.** ClassName::staticMethod
- **B.** obj::instanceMethod
- **C.** ClassName::instanceMethod (arbitrary receiver)
- **D.** ClassName::new

Answers: **A, B, C, D**
Explanation: All four forms exist: static, bound instance, unbound instance, and constructor references.

Q152.
MS

Stream + lambdas: which short-circuit ops? (Select all that apply)

- **A.** anyMatch
- **B.** allMatch
- **C.** noneMatch

- **D.** forEach

Answers: **A, B, C**
Explanation: The match operations can short-circuit; forEach is terminal and processes all elements.

—

20. FileUtils Classes

Q153.
MC

Which library provides FileUtils and IOUtils?

- **A.** Java SE (java.io)
- **B.** Apache Commons IO
- **C.** Google Guava
- **D.** Spring Core

Answer: **B**
Explanation: FileUtils and IOUtils are part of Apache Commons IO library.

Q154.
MS

Select all typical capabilities of FileUtils: (Select all that apply)

- **A.** Copy files/directories
- **B.** Read file to String/byte[]
- **C.** Delete quietly
- **D.** Open JDBC connections

Answers: **A, B, C**
Explanation: FileUtils supports common file operations; JDBC is unrelated.

Q155.
MC

Which method reads a file into a String with a specific charset?

- **A.** Files.readString(Path)
- **B.** FileUtils.readFileToString(File, Charset)
- **C.** IOUtils.toString(InputStream, Charset)
- **D.** FileInputStream.readAll()

Answer: **B**
Explanation: In Commons IO, FileUtils.readFileToString(File, Charset) reads file content as String.

Q156.
MC

What does FileUtils.deleteQuietly(File) do?

- **A.** Deletes and throws IOException on failure
- **B.** Deletes and suppresses exceptions, returning boolean
- **C.** Moves file to trash
- **D.** Securely wipes contents

Answer: **B**
Explanation: deleteQuietly attempts deletion and suppresses exceptions, indicating success via boolean.

Q157.
MS

FileUtils common methods: (Select all that apply)

- **A.** copyDirectory(src,dst)
- **B.** forceMkdir(dir)
- **C.** sizeOfDirectory(dir)
- **D.** openSocket(host,port)

Answers: **A, B, C**
Explanation: FileUtils offers directory copy, mkdir, and size calculations; sockets are unrelated.

Q158.
MC

Writing text with charset:

- **A.** writeStringToFile(file, data, charset)
- **B.** appendString(file) only
- **C.** write(file, CharSequence, charset)
- **D.** Only OutputStream allowed

Answer: **A, C**
Explanation: Commons IO supports writeStringToFile and write with charset to files.

Q159.
MC

listFiles can filter using:

- **A.** IOFileFilter
- **B.** FilenameFilter
- **C.** FileFilter
- **D.** Comparator

Answer: **A, C**
Explanation: FileUtils.listFiles commonly uses IOFileFilter/FileFilter; Comparator isn't a filter.

Q160.
MC

moveFile vs renameTo:

- **A.** FileUtils handles cross-filesystem moves
- **B.** renameTo may fail across devices
- **C.** moveFile can overwrite by default
- **D.** renameTo throws IOException

Answer: **A, B**
Explanation: FileUtils.moveFile performs copy+delete if needed; renameTo may fail across volumes; overwrite depends on method variant.

—

21. IOUtils Classes

Q161.

MS

Which are common uses of IOUtils? (Select all that apply)

- **A.** Copy between streams
- **B.** Convert streams to String
- **C.** Close resources quietly
- **D.** Parse JSON

Answers: **A, B, C**

Explanation: IOUtils provides stream utilities; JSON parsing is not its role.

Q162.

MC

Which method copies all data from an InputStream to an OutputStream?

- **A.** IOUtils.transfer(...)
- **B.** IOUtils.copy(InputStream, OutputStream)
- **C.** Files.copy(InputStream, Path)
- **D.** Stream.copyAll(...)

Answer: **B**

Explanation: IOUtils.copy handles stream-to-stream copy; Files.copy handles stream to Path in NIO.

Q163.

MC

What does IOUtils.closeQuietly(Closeable) do?

- **A.** Closes and rethrows IOException
- **B.** Closes and swallows IOException
- **C.** Flushed only
- **D.** Closes only Readers

Answer: **B**

Explanation: closeQuietly closes and suppresses IOExceptions.

Q164.

MS

Select all that are advantages of IOUtils.toString(InputStream, Charset): (Select all that apply)

- **A.** Simplifies reading stream into String
- **B.** Automatically detects encoding
- **C.** Requires specifying Charset to avoid platform default
- **D.** Works with byte[] only

Answers: **A, C**

Explanation: It reads the entire stream to a String; you should specify Charset to avoid relying on platform default; it doesn't auto-detect encoding.

Q165.

MS

IOUtils methods: (Select all that apply)

- **A.** toByteArray(InputStream)
- **B.** toString(Reader)
- **C.** lineIterator(Reader)
- **D.** copyLarge(InputStream, OutputStream)

Answers: **A, B, C, D**

Explanation: IOUtils provides these convenience methods for streams and readers.

Q166.

MS

Character encodings: (Select all that apply)

- **A.** Always specify Charset to avoid default
- **B.** IOUtils auto-detects BOM and encoding
- **C.** Passing Charset avoids platform surprises
- **D.** Close streams after use

Answers: **A, C, D**

Explanation: You should pass Charset, manage resources, and not rely on auto-detection unless using specific utilities.

Q167.

MC

copy vs copyLarge:

- **A.** copyLarge handles >2GB lengths
- **B.** copy may use int sizes
- **C.** Both close streams automatically
- **D.** Neither closes streams automatically

Answer: **A, B, D**

Explanation: copyLarge is suited for very large streams; both do not close streams automatically.

Q168.

MC

closeQuietly caveat:

- **A.** Swallows IOException
- **B.** Safer to close in try-with-resources
- **C.** Always recommended
- **D.** Works for Closeable

Answer: **A, B, D**

Explanation: It suppresses IOExceptions; prefer try-with-resources today; works with Closeable.

—

22. Random Utility Class

Q169.

MC

Which class provides cryptographically strong random numbers?

- **A.** java.util.Random
- **B.** java.security.SecureRandom
- **C.** Math.random
- **D.** ThreadLocalRandom

Answer: **B**

Explanation: SecureRandom is designed for cryptographic strength; Random and Math.random are not.

Q170.

MS

Select all true about ThreadLocalRandom: (Select all that apply)

- **A.** Optimized for concurrent use
- **B.** Use nextInt(origin, bound) for ranges
- **C.** Seed must be set manually
- **D.** Available since Java 7

Answers: **A, B, D**

Explanation: ThreadLocalRandom avoids contention; supports range methods; doesn't require manual seeding; introduced in Java 7.

Q171.

MC

What does Math.random() return?

- **A.** double in [0.0,1.0)
- **B.** float in (0.0,1.0]
- **C.** int 0 or 1
- **D.** boolean

Answer: **A**

Explanation: Math.random returns a double uniformly distributed in [0.0, 1.0).

Q172.

MS

Which statements about java.util.Random are true? (Select all that apply)

- **A.** Deterministic given seed
- **B.** Thread-safe without contention
- **C.** Provides nextInt(bound)
- **D.** Suitable for cryptographic keys

Answers: **A, C**

Explanation: Random is pseudo-random and deterministic with a given seed; it's not cryptographically secure and not ideal for high-concurrency.

Q173.

MS

Random facts: (Select all that apply)

- **A.** Same seed → same sequence
- **B.** Math.random() uses a Random under the hood
- **C.** SecureRandom is slower but stronger
- **D.** ThreadLocalRandom.current() used in parallel contexts

Answers: **A, B, C, D**
Explanation: Seeding repeats sequences; Math.random delegates to a Random; SecureRandom is for crypto; ThreadLocalRandom avoids contention.

Q174.
MC

ThreadLocalRandom:

- **A.** Cannot be constructed directly
- **B.** Use ThreadLocalRandom.current()
- **C.** Methods for bounded ints exist
- **D.** Shares state across threads

Answer: **A, B, C**
Explanation: You obtain it via current(); each thread has its own instance.

Q175.
MC

nextInt(5) returns values in:

- **A.** [0,5)
- **B.** (0,5]
- **C.** [1,5]
- **D.** [0,4]

Answer: **D**
Explanation: Bounded nextInt is exclusive upper bound.

Q176.
MS

SecureRandom seeding: (Select all that apply)

- **A.** May block until enough entropy
- **B.** Can be seeded manually
- **C.** Is reproducible by default
- **D.** Used for keys/tokens

Answers: **A, B, D**
Explanation: SecureRandom may block, can be seeded, and is suitable for cryptographic material.

—

23. Scanner Utility Class

Q177.
MC

Which package contains Scanner?

- **A.** java.io
- **B.** java.util
- **C.** java.nio
- **D.** java.text

Answer: **B**
Explanation: Scanner is in java.util.

Q178.
MS

Select all correct about Scanner tokenization: (Select all that apply)

- **A.** Default delimiter is whitespace
- **B.** You can set a custom delimiter with useDelimiter
- **C.** nextInt() skips delimiters
- **D.** nextLine() returns the rest of the current line, excluding line separator

Answers: **A, B, C, D**
Explanation: Scanner splits input by delimiter (default whitespace); nextInt skips delimiters and parses; nextLine returns remaining line without separator.

Q179.
MC

Which methods would you use to read a full line and an integer?

- **A.** next() and nextFloat()
- **B.** nextLine() and nextInt()
- **C.** readLine() and parseInt()
- **D.** getline() and stoi()

Answer: **B**
Explanation: Use nextLine for full line (including spaces) and nextInt for integers.

Q180.
MC

What is a common pitfall when mixing nextInt() and nextLine()?

- **A.** nextInt() consumes the newline, so nextLine() reads the next token
- **B.** nextInt() leaves the newline, so nextLine() reads an empty line
- **C.** They are incompatible
- **D.** nextLine() reads integers automatically

Answer: **B**
Explanation: nextInt() leaves the newline in the buffer; calling nextLine() immediately after may read an empty line unless you consume the newline.

Q181.
MS

Locale effects: (Select all that apply)

- **A.** Decimal separator depends on Locale

- **B.** setLocale affects number parsing
- **C.** US vs EU decimal comma/point differences
- **D.** No effect on parsing

Answers: **A, B, C**
Explanation: Scanner respects Locale for numbers; e.g., comma vs dot.

Q182.
MC
hasNextInt vs nextInt:

- **A.** hasNextInt checks without consuming
- **B.** nextInt parses and consumes
- **C.** hasNextInt changes delimiter
- **D.** nextInt ignores delimiter

Answer: **A, B**
Explanation: hasNextInt is a predicate; nextInt consumes the token.

Q183.
MC
Closing a Scanner on System.in:

- **A.** Also closes System.in
- **B.** Is harmless always
- **C.** May break subsequent input
- **D.** Recommended always

Answer: **A, C**
Explanation: Closing it closes underlying stream; often avoid closing System.in.

Q184.
MC
useDelimiter takes:

- **A.** A regex
- **B.** A literal only
- **C.** A char array
- **D.** A Predicate

Answer: **A**
Explanation: useDelimiter accepts a regex to split tokens.

—